ESPRIT School of Business

Licence 3 — Business Computing BI

Projet Intégré

Analyse et Prédiction du Churn Client

Guide Étudiant — Démarche et Outils par Étape

Tuteurs du projet

Aymen Ben Brik et Hela Hammami

2026

Table des matières

Étape 0 — Mise en place de l'environnement **2**
Objectifs . . . 2
À faire . . . 2
Outils proposés . . . 2

Étape 1 — Collecte et compréhension des données (semaine du 9/11) **2**
Obligatoire . . . 2
Bonus . . . 3
Outils proposés . . . 3

Étape 2 — Conception du DWH et ETL (semaine du 16/11) **3**
Obligatoire : Talend + PostgreSQL / pgAdmin . . . 3
Bonus . . . 3
Outils proposés . . . 4

Étape 3 — Reporting et dashboard (pour le 23/11) **4**
Obligatoire : Power BI + DAX . . . 4
Bonus . . . 4

Étape 4 — Modèles prédictifs (pour le 1/12) **4**
4.1 Obligatoire : fixer un KPI et prévoir sa série temporelle . . . 4
4.2 Obligatoire : Machine Learning pour le churn . . . 5
Bonus . . . 5
Outils proposés . . . 5

Étape 5 — Déploiement (pour le 7/12) **5**
Obligatoire : interface web . . . 5
Obligatoire : diagrammes UML . . . 6
Bonus . . . 6
Outils proposés . . . 6

Étape 6 — Rapport et présentation **6**
Objectifs . . . 6
Structure recommandée du rapport (25–40 pages) . . . 6
Outils proposés . . . 7
Conseils pour la présentation orale . . . 7

Récapitulatif — Stack du projet **7**

Ce guide vous accompagne **étape par étape** dans la réalisation du projet. Pour chaque phase, vous trouverez :

— Le socle **obligatoire** à réaliser et son échéance.
— Les **outils imposés** et les **bonus**, avec une proposition d'outils pour le reste (justifiez vos choix dans le rapport).

> **Conseil général** : traitez d'abord tout ce qui est **obligatoire**. Les bonus ne rapportent des points que si le socle est solide.

Étape 0 — Mise en place de l'environnement

Objectifs

— Préparer un environnement de travail reproductible.
— Mettre en place le dépôt GitHub et la collaboration.

À faire

— Créer un dépôt **GitHub** sans aucune donnée.
— Inviter les tuteurs en tant que collaborateurs.
— Créer un environnement virtuel Python (`venv` ou `conda`) et rédiger un `requirements.txt`.
— Rédiger un premier `README.md` : description, structure du dépôt, instructions d'installation.
— Créer un `.gitignore` qui exclut **toutes les données** (`data/`, `*.csv`, `*.xlsx`, `*.pkl`, `*.pbix`) et les fichiers d'environnement (`venv/`, `__pycache__/`, `.env`).

Outils proposés

Catégorie	Options
Versioning	**Git + GitHub** (obligatoire)
IDE	VS Code, PyCharm
Notebooks	Jupyter Lab, VS Code Notebooks
Gestion de projet	GitHub Projects, Trello, Notion

Étape 1 — Collecte et compréhension des données (semaine du 9/11)

Obligatoire

— Charger les fichiers, vérifier le grain et les clés, croiser avec les fichiers de référence.
— **Analyse univariée** : distributions, indicateurs de position et de dispersion, modalités dominantes.
— **Décompte des valeurs manquantes** par variable, en distinguant les vides métier des vides d'anonymisation quand c'est possible.
— **Décompte des outliers** par variable numérique, avec la méthode utilisée (IQR, z-score). Interprétez avant de supprimer : un solde négatif peut être normal.

Bonus

— **Analyse bivariée** : variable explicative × churn, corrélations, tests statistiques.
— **Web scraping** avec **Beautiful Soup** (pages statiques), **Playwright** (pages dynamiques) ou **Scrapy** (collecte structurée).

> **Règles du web scraping** : collectez uniquement des informations publiques et générales sur le secteur bancaire ou le churn. Respectez les conditions d'utilisation et `robots.txt` des sites. Il est interdit de chercher à relier ces données au fichier fourni ou à identifier l'établissement ou un client. Les dates étant décalées, aucune jointure calendaire n'a de sens.

Outils proposés

Catégorie	Options
Manipulation	**pandas**, **polars**, **numpy**
Visualisation	**matplotlib**, **seaborn**, **plotly**
Profilage automatique	`ydata-profiling`, en local uniquement

Étape 2 — Conception du DWH et ETL (semaine du 16/11)

Obligatoire : Talend + PostgreSQL / pgAdmin

1. **Conception** : définir le grain, la table de faits et les dimensions du modèle en étoile.
2. **Base cible** : créer la base et le schéma sous **PostgreSQL** avec **pgAdmin**.
3. **Extract** : lire le CSV et les fichiers Excel dans **Talend**.
4. **Transform** : convertir les dates `YYYYMMDD`, traiter les vides et les doublons, joindre les fichiers de référence, calculer les variables dérivées.
5. **Load** : charger les dimensions, puis les faits, en contrôlant les clés.
6. **Contrôle** : comparer les volumes et les indicateurs entre la source et l'entrepôt.

Bonus

Bonus	Attendu
SSIS	Reproduire un flux et comparer avec Talend
Databricks	Architecture médaillon : **bronze** (brut), **silver** (nettoyé), **gold** (prêt pour l'analyse)
SCD de type 2	Historiser une dimension avec dates de validité et indicateur de version courante
Python	Pipeline reproductible (pandas, SQLAlchemy)

> **Important** : le pipeline doit être **reproductible** et **documenté** par un schéma clair du flux. Exportez les jobs Talend dans le dépôt, sans les fichiers de données. Si vous utilisez Databricks, n'y chargez les données qu'avec l'accord des tuteurs.

Outils proposés

Catégorie	Options
ETL	**Talend** (obligatoire)
Base de données	**PostgreSQL + pgAdmin** (obligatoire)
Schéma	dbdiagram.io, draw.io, Mermaid

Étape 3 — Reporting et dashboard (pour le 23/11)

Obligatoire : Power BI + DAX

— Connecter **Power BI Desktop** à PostgreSQL.
— Définir les KPI en **DAX** : taux de churn, ancienneté moyenne, solde moyen, etc. Attention au grain : un taux calculé par ligne n'est pas un taux par compte.
— Construire au minimum **3 pages** :

1. **Vue d'ensemble** : KPI clés, taux de churn global, évolution.
2. **Analyse du churn** : décomposition par segment (âge, secteur, ancienneté, produit, devise).
3. **Segmentation client** : profils et valeur client.

— Soigner la mise en page : filtres, navigation entre pages, infobulles.

Bonus

— **Visualisation avec Databricks**.
— **Interface JavaScript** (par exemple Chart.js, D3.js ou React) alimentée par des agrégats.

> **Conseil** : la qualité du design compte, mais la justesse des mesures compte davantage. Ne publiez pas le rapport sur le Web : un `.pbix` contient des données. Remettez-le aux tuteurs hors GitHub.

Étape 4 — Modèles prédictifs (pour le 1/12)

4.1 Obligatoire : fixer un KPI et prévoir sa série temporelle

1. **Choisir un KPI** utile au métier, par exemple le nombre hebdomadaire de clôtures de comptes ou le taux de clôture.
2. **Construire la série** au grain compte, en semaines, en ajoutant les semaines à zéro.
3. **Analyser** : tendance, saisonnalité, autocorrélation (ACF/PACF), stationnarité, pics.
4. **Traiter les pics** : cinq semaines concentrent environ la moitié des clôtures. Identifiez-les et justifiez votre traitement.
5. **Modéliser** : entraîner un **LSTM** ou un **RNN** sur des fenêtres glissantes, avec normalisation calculée uniquement sur l'entraînement.
6. **Comparer** à une référence simple (prévision naïve ou saisonnière, éventuellement ARIMA). Le LSTM doit prouver son intérêt : avec environ 200 points, il peut ne pas gagner, et ce résultat est acceptable s'il est démontré.

7. **Évaluer** sur la fin de la série, sans mélange temporel, avec MAE et RMSE. Le MAPE n'est pas défini sur les semaines à zéro.

4.2 Obligatoire : Machine Learning pour le churn

1. **Préparer** : fixer le grain, sélectionner les variables et écarter celles qui provoquent une **fuite de cible** (document 2, section 5.1).
2. **Modéliser** : comparer **au moins 3 modèles** (par exemple régression logistique, Random Forest, XGBoost).
3. **Découper sans contamination** : un même client ne doit pas se trouver à la fois dans l'entraînement et le test (`GroupShuffleSplit`).
4. **Évaluer** : precision, recall, F1, **ROC-AUC**, **PR-AUC**, matrice de confusion. Pas seulement l'accuracy.
5. **Optimiser et interpréter** : hyperparamètres du meilleur modèle, importance des variables, valeurs SHAP.
6. **Sauvegarder** le modèle final hors du dépôt.

Bonus

— Traiter **plusieurs KPI**, avec un modèle par KPI ou un modèle multivarié.

Outils proposés

Catégorie	Options
Séries temporelles	**statsmodels**, **pandas**
Deep Learning	**TensorFlow / Keras**, **PyTorch**
Machine Learning	**scikit-learn**, **XGBoost**, **LightGBM**
Hyperparamètres	`GridSearchCV`, `RandomizedSearchCV`, **Optuna**
Interprétabilité	**SHAP**, importance native des arbres
Sauvegarde	**joblib**

Étape 5 — Déploiement (pour le 7/12)

Obligatoire : interface web

— Concevoir une interface simple avec au moins :

  — un **formulaire de prédiction** : saisie d'un profil fictif → score de churn ;
  — l'**affichage de la prévision du KPI** ;
  — des **KPI de synthèse** agrégés.

— Déployer sur une plateforme accessible et donner l'URL dans le README.

> **Protection des données** : une application publique n'affiche ni ligne client, ni identifiant, ni export. Le dépôt et l'image déployée ne contiennent aucune donnée. Si vous présentez une liste de comptes à risque, générez-la sur des **profils fictifs** ou placez-la derrière une authentification, sur un serveur non public.

Obligatoire : diagrammes UML

Diagramme	Attendu
Cas d'utilisation global	Acteurs et fonctionnalités de l'application
Séquence (un seul)	Le scénario principal, par exemple la demande de prédiction
Classes	Structure de l'application et liens entre les classes

Bonus

— **DevOps** : tests automatisés (`pytest`) et intégration continue (GitHub Actions).
— **Application mobile** : Flutter, React Native ou PWA, avec les mêmes règles de protection des données.

Outils proposés

Catégorie	Options
Application data	**Streamlit**, **Gradio**, **Dash**
Front + API	**FastAPI** ou Flask avec React, Vue ou Bootstrap
Hébergement	Streamlit Community Cloud, Hugging Face Spaces, Render
UML	**draw.io**, PlantUML, Mermaid, StarUML

Étape 6 — Rapport et présentation

Objectifs

— Documenter votre démarche, vos choix et vos résultats.
— Convaincre le jury en 20 minutes que votre solution répond au besoin.

Structure recommandée du rapport (25–40 pages)

1. **Page de garde et résumé exécutif** (1 page).
2. **Introduction et contexte** (2 pages).
3. **Compréhension des données** (3–5 pages : univarié, manquants, outliers).
4. **Architecture de la solution** (2–3 pages, avec schémas).
5. **DWH et ETL** (3–5 pages : modèle en étoile, Talend, PostgreSQL).
6. **Reporting Power BI et DAX** (3–5 pages, captures sans ligne client).
7. **Modèles prédictifs** (5–8 pages : KPI, série temporelle, LSTM/RNN, churn, contrôle de la fuite de cible).
8. **Interface web, déploiement et UML** (2–3 pages, captures, diagrammes, lien démo).
9. **Limites et perspectives** (1–2 pages).
10. **Conclusion** (1 page).
11. **Annexes** : références, code clés, glossaire.

Outils proposés

Catégorie	Options
Rédaction du rapport	**LaTeX** (Overleaf), **Microsoft Word**, **Google Docs**, **Typst**, Markdown + Pandoc
Diagrammes	**draw.io**, **Lucidchart**, **Mermaid**, **Excalidraw**, dbdiagram.io
Captures d'écran	ShareX, Snipping Tool, macOS Screenshot
Slides	**PowerPoint**, **Google Slides**, **Canva**, **Beamer** (LaTeX), Marp (Markdown)

Conseils pour la présentation orale

— **Temps** : 20 minutes de présentation + 10 minutes de questions. Ne dépassez pas.
— **Rythme** : ~1 slide par minute.
— **Démo** : prévoyez **au moins 5 minutes** de démonstration live de votre application web. Préparez un plan B (vidéo de secours) en cas de problème de connexion.
— **Répartition** : chaque membre de l'équipe doit prendre la parole.
— **Public** : présentez à un public à la fois technique et métier — expliquez les choix techniques de manière accessible.

Récapitulatif — Stack du projet

Composante	Obligatoire	Bonus possibles
Compréhension des données	Python, pandas, Jupyter	Scraping, analyse bivariée
ETL	**Talend**	SSIS, Databricks, Python
Data Warehouse	**PostgreSQL + pgAdmin**	Médaillon, SCD de type 2
Reporting	**Power BI + DAX**	Databricks, JavaScript
Série temporelle	**LSTM / RNN**	Plusieurs KPI
Churn	Machine Learning (scikit-learn, XGBoost)	Plusieurs KPI
Interface	Application web + **UML**	Mobile, tests DevOps
Versioning	**Git + GitHub**	GitHub Actions